Study report : TP2 - CNN and Transfer Learning : Age Estimation form Frace Images

Dataset: APPA-REAL subset — 3,999 train / 1,199 validation / 1,200 test images. Training loss: MSE. Reported metric: MAE (Mean Absolute Error), because MSE is in squared years and is hard to interpret. MAE says "the model is wrong by X years on average".

Part 1 - A CNN from Scratch

Test MAE and usefulness. The best weights come from epoch 19 (val MAE = 11.68). The **test MAE is 14.03 years**. A trivial **baseline** that always predicts the mean training age (30.4 years) gets 14.39 years. So the CNN is only 0.36 years better than the baseline: it is **not useful** in practice.

Overfitting or underfitting? The train and val MAE curves are almost on top of each other (gap < 0.5 years). So there is **no overfitting**. But both errors stay high (≈ 11.5–12.5 years) and reach a plateau quickly. The model cannot learn well, even on the training data: this is **underfitting**. With only ~4,000 images and no **pretrained features**, a small CNN cannot learn facial features that are good enough to estimate age.

Peaks in the curves. We see error peaks at epochs 5, 13 and 18 (val MAE = 16.4 at epoch 18). Train MAE and val MAE jump **at the same time**, while the training MSE loss keeps going down smoothly. So the peaks come from the state of the model, not from overfitting. Possible causes: a **learning rate** a bit too high for **Adam** (lr = 1e-3), and **BatchNorm** statistics in eval mode. This **training instability** also shows that the model has not finished learning.

Part 2 - Transfer Learning with ResNet-18

Stage 1 - Feature extraction (frozen backbone)

Trainable-parameter ratio. Only **513 of 11,177,025** parameters are trainable (≈ 0.005 %). This is what we expect. A standard ResNet-18 has ~11.69 M parameters, and its original ImageNet head `Linear(512, 1000)` has 513,000 of them. We replace it with `Linear(512, 1)` : 11,689,512 − 513,000 + 513 = 11,177,025. Only the 512 weights + 1 bias of the new head are trained. The model is just a **linear regression** on the 512 features of the pretrained **backbone**.

Stage 2 - Fine-tuning: why lr = 1e-4?

Stage 1 uses lr = 1e-3; Stage 2 uses **lr = 1e-4** (10× smaller). Now all 11.2 M parameters are updated. The ImageNet filters are already good. A large learning rate would make big updates and **destroy these pretrained features** (**catastrophic forgetting**). A small learning rate only **adjusts** the features slightly for the new task (faces and age).

Experiment 1 - Comparing the Three Approaches

Ranking.

1. Fine-tuned ResNet-18: **8.78 years**
2. Frozen ResNet-18: **12.52 years**
3. From-scratch CNN: **14.03 years** This ranking matches the **transfer learning** theory from CM3. With a small dataset, starting from a network **pretrained on a large dataset** (ImageNet) is much better than learning the filters from zero. Full fine-tuning adapts the pretrained features to the target task and reduces the error by **37 %** compared with the from-scratch CNN.

Stage 1 (frozen) vs. Stage 2 (fine-tuned). Unfreezing **clearly helped**: test MAE goes from 12.52 to 8.78 years (−3.7 years, −30 %). The gain is visible from the first fine-tuning epoch (val MAE 10.58 -> 8.39). The frozen model was

underfitting: a linear model on features made to classify objects cannot capture the fine details of age.

But fine-tuning also shows the **risk** of training a large network on a small dataset. There are 11.2 M parameters for ~4,000 images (≈ 2,800 parameters per image), so the network **memorises** the training set quickly. Train MAE goes below 2 years while val MAE stays around 7 years: a **gap of ~5 years** (**overfitting**). This gap did not exist with the frozen backbone. Here, the low learning rate and the **best-checkpoint selection** keep the result good. With an even smaller dataset, it would be safer to unfreeze only the last layers (layer4) and keep the early layers frozen.

Why do object features transfer to faces? A CNN learns a **hierarchy of features**:

- **Early layers** detect generic patterns: edges, orientations, colours, simple textures. These exist in all natural images.
- **Middle layers** combine them into shapes and object parts (curves, contours, repeated patterns).
- Only the **last layers** are specific to the 1,000 ImageNet classes. Many **age cues** are low- and mid-level features:
- **textures:** wrinkles, skin grain, spots;
- **colours:** grey or white hair, skin tone;
- **shapes and proportions:** round face of a child, eye size vs. head size, jaw line. Also, ImageNet contains many images with people in them, so the network has already seen faces, skin and hair. The transfer is **not perfect**: high-level features are made for objects, not age. This is why the frozen backbone gives only a small gain (12.52 years), and fine-tuning, which mostly adapts the deep layers, gives most of the improvement.

Experiment 2 - Does Data Augmentation Help Fine-tuning?

Test MAE. With **data augmentation** (random horizontal flip, small rotation, colour jitter), the fine-tuned ResNet gets **8.63 years**, vs. 8.78 without. The gain is only **0.15 years (≈ 1.7 %)**. This is smaller than the normal **run-to-run variability** (random head initialisation, batch order, random augmentations). With only one run per configuration, we **cannot conclude** that augmentation really helps: at best, the effect is small. In Stage 1 (frozen), augmentation even gives a slightly worse val MAE (10.77 vs. 10.58). This makes sense: with 513 parameters the model already underfits, and harder training images do not help.

Train/val gap. Augmentation **reduced the gap but did not close it**: the MAE gap goes from ~5.1 to ~4.2 years. The training loss also decreases much more slowly (final train MSE ≈ 20 vs. ≈ 10 without augmentation), while the val MSE stays at the same level. This is consistent with augmentation acting as a **regulariser**: the model never sees the exact same image twice, so it is harder to memorise. But it **did not improve generalisation**: val MAE stays around 7.1–7.3 years. So overfitting was **not the main limiting factor** here; pretraining, the low learning rate and checkpoint selection already controlled it.

Best val MAE vs. test MAE. The two sets give **opposite rankings**: on validation, augmentation is slightly worse (7.12 vs. 7.06); on test, it is slightly better (8.63 vs. 8.78). Both differences (0.06 and 0.15 years) are tiny. They are smaller than the **measurement noise**: with ~1,200 validation images and an absolute error of ~7 years per image (with a large spread), the **statistical uncertainty** on the mean MAE is about **0.2 years**. We also see it in the curves: val MAE moves by 0.1–0.3 years from one epoch to the next without a real change in the model. So **val MAE alone is not a reliable signal** for small effects like augmentation. It can detect large differences (scratch vs. fine-tuning, > 5 years), but not differences of a few tenths of a year. Several runs (different seeds) or a larger validation set would be needed.

Experiment 3 - How Much of the Network Should You Fine-tune?

Position of partial fine-tuning. Partial fine-tuning (layer4.1 + fc) gets a **test MAE of 10.71 years**. It is **between** the frozen backbone (12.52) and full fine-tuning (8.78), roughly in the middle: it gets about **half of the gain** of full fine-tuning (1.8 of 3.7 years).

Was the trade-off worth it? No, not here. It trains 42 % of the parameters (4.7 M, because layer4 holds most of ResNet-18's parameters) for only ~50 % of the gain, so the cost saving is limited. Even more important, partial fine-tuning **overfits as much as (or more than)** full fine-tuning: train MAE ≈ 2.4 years, val MAE stuck at 9.1–9.7 years, a **gap of ~7 years** (vs. ~5 for full FT). Training fewer parameters did **not** reduce overfitting. A possible explanation: only the last block adapts to faces, and it works on intermediate features (layer1 to layer4.0) that are still made for ImageNet objects. So it mainly learns to memorise the training set from these features. Full fine-tuning also adapts the middle layers (skin texture, face shape), which **generalises better**. With ~4,000 images, **full fine-tuning is the best choice**.

With only 500 images? Full fine-tuning would look **relatively worse** compared with partial fine-tuning. It would train 11.2 M parameters for 500 images (> 22,000 parameters per image, vs. ~2,800 now). The **overfitting risk**, already visible with 4,000 images, would become much larger: the network could memorise the 500 images and move away from the good ImageNet features. With less data, it is better to **train fewer parameters**: unfreeze only the last layers, or only the head. The frozen layers keep the general ImageNet features and act as a **protection against overfitting**. The smaller the dataset, the more of the network we should freeze. But our experiment shows that freezing alone is not enough: with 500 images, we should also add **regularisation** (data augmentation, weight decay) and use **fewer epochs / early stopping**.

Overall Comparison

Ranking and effect of augmentation.

1. Full FT + augmentation: 8.63 years
2. Full FT: 8.78 years
3. Partial FT: 10.71 years
4. Frozen: 12.52 years
5. From scratch: 14.03 years Augmentation does **not** give the biggest gain: only 0.15 years on top of full fine-tuning. Going from frozen to full fine-tuning gives **3.74 years**, about **25× more**. Even each intermediate step (scratch -> frozen: 1.51; frozen -> partial: 1.81; partial -> full: 1.93) is more than 10× larger than the augmentation effect. The order of the top two is **not robust** (on validation, full FT without augmentation is better: 7.06 vs. 7.12). The **key factor** is how much of the pretrained network we allow to adapt, not augmentation.

Best epoch column. All models reach their best val MAE **late**, between epochs 16 and 19 out of 20. None peaks early and then gets worse. So the **20-epoch budget is about right**, with some differences:

- **From scratch (19):** very noisy curve with peaks; epoch 19 is mostly a "lucky" epoch. More epochs would not help much: the problem is underfitting, not training time.
- **Frozen (16): plateau** from about epoch 12 (10.58–10.80). A few more epochs or a higher lr would give only a marginal gain.
- **Full FT (19) and partial FT (18):** val MAE keeps decreasing slowly until the end (full FT: 7.52 at epoch 10 -> 7.06 at epoch 19). They use the full budget and could gain a few tenths of a year with more epochs.
- **Full FT + aug (16):** plateau around 7.1–7.3 from about epoch 14. Augmentation slows memorisation but does not extend progress on validation. In practice, most of the gain is obtained in the **first 5–10 epochs** for all methods. After that, epoch-to-epoch changes (0.1–0.3 years) are at the level of the noise, so the exact "best epoch" is not very meaningful. Note that fine-tuning methods actually train for **40 epochs** in total (20 for Stage 1 + 20 for Stage 2).

Best accuracy / training-cost trade-off. The number of trainable parameters gives an idea of the cost, but all ResNet variants do the same full **forward pass**. Only **backpropagation** changes (it stops at fc for frozen, at layer4.1 for partial, and goes through the whole network for full FT).

- **Frozen:** by far the cheapest, best gain per parameter (1.87 years better than the baseline with 513 parameters), but accuracy is not good enough (12.5 years).
- **Partial FT:** the least interesting: 42 % of the parameters for half the gain, same overfitting, limited saving.
- **Full FT (no augmentation): the best trade-off**. Best accuracy (3.7 years better than frozen, 1.9 better than partial), and the extra cost is small in absolute terms on ~4,000 images (a few more minutes on GPU).
- **Full FT + augmentation:** same parameters plus on-the-fly transforms, for a non-significant 0.15-year gain. Not justified here, but still a good habit with less data or more epochs. **Conclusion:** for a dataset of this size, **full fine-tuning with a low learning rate** gives the best accuracy-per-cost ratio.

Experiment 4 - Feature Map Visualisation

Do channels respond to different things? Yes. The channels of the early convolutional layer react to clearly different patterns: some detect **oriented edges**, some detect **fine textures**, others respond to specific **face regions** or to the global shape of the image. A few channels are almost **inactive**, because the pattern they detect is not present in this image. This diversity shows that the first layer **decomposes the image** into several complementary visual "building blocks".

- **Channel 5 (row 1, image 6) - edge detector.** It is dark almost everywhere and only activates along curved, mostly vertical lines that follow the outline of the face and hair. It reacts to **sharp intensity changes** (edges) and ignores uniform areas like skin or background.
- **Channel 11 (row 2, image 4) - texture detector.** It activates in a scattered way over the whole image, as many small, fine, repeated patterns, without following a contour. It reacts to **high-frequency variations** like skin grain, hair or background details. This is relevant for age, because **wrinkles and skin texture** change with ageing.
- Also, **channel 4 (row 1, image 5)** reproduces the face almost exactly: it responds to **global intensity**, separating dark areas (eyes, mouth) from bright areas. These are **low-level, generic features**: the same kind of detectors as those learned on ImageNet. This explains why they **transfer well** to faces.